

Day 76: Basic ML Preprocessing — Encoding and Scaling

1. Day Number

Day 76

2. Topic Name

Categorical Encoding, Scaling, and Normalization

Today you will learn how to transform data into forms that are easier for machine-learning models to work with.

3. Connection to Yesterday

Yesterday, in **Day 75**, you cleaned raw data by dealing with problems such as missing values, duplicates, and incorrect data types.

Today, you move one step further:

Raw data → Clean data → Preprocessed data → Machine-learning model

Cleaning fixes problems in the data. **Preprocessing transforms valid data into a form that is more suitable for machine learning.**

4. Important Topics

The main ideas for today are:

- **Numerical feature** — a column containing numbers, such as age, salary, price, or temperature.
 - **Categorical feature** — a column containing categories or labels, such as city, color, or product type.
 - **Encoding** — converting categories into numeric representations.
 - **Scaling** — changing numerical values so they are on a more comparable scale.
 - **Standardization** — transforming values based on their average and spread.
 - **Normalization** — commonly transforming values into a fixed range such as 0 to 1.
-

5. Foundational Notes

In machine learning, a **feature** is usually an input value used by a model.

Imagine this tiny dataset:

Age	City
20	Delhi
30	Mumbai

Age	City
40	Delhi

Here:

- Age is a **numerical feature**.
- City is a **categorical feature**.

Before giving this data to many machine-learning algorithms, we may need to transform both columns.

For example:

```
City
Delhi
Mumbai
Delhi
```

could be encoded conceptually as:

```
City_Code
0
1
0
```

The exact encoding method depends on the problem. For now, focus only on the basic idea that **text categories can be represented numerically**.

6. Why Can't a Model Always Directly Use Text Categories?

Computers can store text, but many traditional machine-learning algorithms mainly perform calculations using **numbers**.

Suppose a column contains:

```
Delhi
Mumbai
Bengaluru
```

A mathematical model cannot usually perform useful calculations such as:

```
Delhi * 0.5
```

Therefore, we transform categories into numeric representations.

For a very simple learning example:

```
Delhi      → 0
Mumbai     → 1
Bengaluru  → 2
```

One important warning: these numbers should **not automatically be interpreted as rankings**.

For example:

```
Delhi = 0
Mumbai = 1
Bengaluru = 2
```

does **not** mean Bengaluru is "twice" Mumbai or that one city is greater than another.

That is why real ML projects choose encoding methods according to what the categories actually mean.

7. Scaling vs Normalization

Both ideas change numerical values, but they do it differently.

Scaling / Standardization Idea

Suppose two features are:

```
Age:      20, 30, 40
Salary: 30000, 60000, 90000
```

The salary numbers are much larger than the age numbers.

Some machine-learning algorithms can become strongly affected by features with larger numerical scales.

Standardization transforms the values according to their average and how spread out they are.

After standardization, values may look roughly like:

```
-1.2
0.0
1.2
```

Negative values are completely normal.

You do **not** need to calculate the formula manually today. Later, libraries such as `scikit-learn` can handle this transformation.

Normalization Idea

Normalization commonly means converting values into a fixed range.

For example:

```
Original ages:  
20, 30, 40  
  
Normalized idea:  
0.0, 0.5, 1.0
```

So:

Standardization: centers/scales values based on their distribution.

Normalization: often places values inside a particular range such as 0 to 1.

A useful beginner memory trick is:

```
Standardization → relative to average/spread  
Normalization   → usually fixed range
```

Don't worry about the mathematics yet.

8. Easy Examples

Example A — Categorical Encoding

Suppose you have:

```
City  
Delhi  
Mumbai  
Delhi  
Mumbai
```

A simple encoded representation might be:

```
Delhi → 0  
Mumbai → 1
```

giving:

```
0  
1  
0  
1
```

Example B — Normalizing a Numerical Feature

Suppose:

```
Age
20
30
40
```

A simple 0–1 transformation could produce something like:

```
0.0
0.5
1.0
```

The smallest value becomes close to 0, while the largest becomes close to 1.

Example C — Why Scaling Can Matter

Imagine:

```
Age = 25
Income = 80000
```

Numerically, 80000 is enormously larger than 25.

That doesn't necessarily mean income should be thousands of times more important to the model. Scaling can help put numerical features on more comparable scales.

9. Problem Statement

Create a tiny dataset containing **age** and **city**.

For example:

	Age	City
	20	Delhi
	30	Mumbai
	40	Delhi
	50	Bengaluru

Your program should:

1. Store the dataset in a small Pandas DataFrame.
2. Identify Age as a numerical feature.
3. Identify City as a categorical feature.

4. Convert the city categories into a simple numeric representation.
5. Apply one simple scaling technique to the age column.
6. Display the original data and the transformed result.

Keep the exercise very small. You are learning the **preprocessing idea**, not building an ML model yet.

10. Concepts Used

You will practice:

```
Pandas DataFrame
numerical features
categorical features
encoding
category-to-number transformation
scaling
normalization/standardization concept
preprocessing
```

You may later encounter tools such as:

```
LabelEncoder
StandardScaler
MinMaxScaler
```

from `scikit-learn`.

For today, understand what these kinds of tools **do** rather than memorizing every option.

11. Thought Process

When preprocessing a dataset, think through it in this order:

```
What columns do I have?
```

```
↓
```

```
Which columns contain numbers?
```

```
↓
```

```
Which columns contain categories/text?
```

```
↓
```

```
Does a categorical column need numeric encoding?
```

↓

Do numerical columns have very different scales?

↓

Choose a simple transformation.

↓

Transform the data.

↓

Inspect the result.

For today's dataset:

Age → numerical → consider scaling

City → categorical → needs encoding

A good habit is to understand the **meaning of a column before transforming it**.

12. Beginner-Friendly Pseudocode

START

import the required libraries

create a small DataFrame

include age

include city

print the original DataFrame

identify city as categorical

create a simple encoder

fit the encoder using the city column

transform city into numeric values

identify age as numerical

create a simple scaler

reshape/select age in the form expected by the scaler

```
fit the scaler
transform age

store the transformed values

print the transformed DataFrame

END
```

Notice the common pattern:

```
create transformer
    ↓
learn information from data
    ↓
transform data
```

You will see this pattern frequently in machine learning.

13. Suggested Solving Approach — Simple Preprocessing

A beginner-friendly approach is:

```
Pandas
    ↓
Create DataFrame
    ↓
Inspect columns
    ↓
Encode City
    ↓
Scale Age
    ↓
Inspect transformed data
```

Don't build a prediction model yet.

Focus only on answering:

"How can I transform these features into ML-friendly numerical data?"

14. Easy Edge Cases

Unknown Category

Imagine your encoder learned only:

```
Delhi
Mumbai
Bengaluru
```

Later it receives:

```
Chennai
```

The preprocessing logic may not know how to handle this new category.

This is called an **unknown or unseen category**.

For now, simply remember:

| Real ML preprocessing must decide what to do when new categories appear.

You will learn better encoding strategies later.

Constant Numeric Values

Suppose every person's age is:

```
30
30
30
30
```

There is no numerical variation.

Some scaling calculations become unusual because the minimum and maximum are identical, or the spread is zero.

Good ML libraries usually handle these situations safely, but it is important to recognize that the feature contains **no variation**.

15. Hint Only

For the categorical column, think about using a simple encoder:

```
from sklearn.preprocessing import LabelEncoder
```

For the numerical column, investigate **one** of these:

```
StandardScaler
```

or:

```
MinMaxScaler
```

A scaler generally expects numerical data in a **2-dimensional column-like form**, rather than a plain one-dimensional Series.

Try to build this transformation:

Before:

Age	City
20	Delhi
30	Mumbai
40	Delhi
50	Bengaluru

After:

Age_Scaled	City_Encoded
?	?
?	?
?	?
?	?

Your goal is to figure out the missing transformed values with Python rather than manually calculating everything.

Day 76 takeaway: cleaning makes data **correct and usable**, while preprocessing makes data **more suitable for a machine-learning algorithm**.